

Crypto16

Coprocessor



A 16-Bit Hardware Datapath for Cryptographic Operations

Course: CSE 226 — Hardware Design | Zagazig University — Faculty of Engineering

- Ahmed Mahmoud Abdel Fattah
- Ahmed Abdel Fattah Elmallah
- Ahmed Tharwat Salah
- Ibraheem Shaaban Abdel Nabi
- Ahmed Salah Abdel Majeed
- Wateen Walid Mohamed Baree

Agenda

01

Problem Statement

Why do we need a crypto coprocessor?

02

System Architecture

Big picture — how all blocks connect

03

Control Unit

The brain: decoding 4-bit CTRL signals

04

ALU

7 arithmetic & logic operations

05

Shifter

ROR8, ROR4, SLL8 transformations

06

Nonlinear LUT (S-Box)

The heart of cryptographic security

07

Combinational Block

MUX, result selection & integration

08

Simulation & Testing

Verification strategy & waveforms

Why a Crypto Coprocessor?



General CPUs Are Slow

Standard processors handle encryption in software — slow, power-hungry, and exposed to timing attacks.



Security Needs Hardware

Dedicated hardware keeps keys isolated, executes in constant time, and resists side-channel attacks.

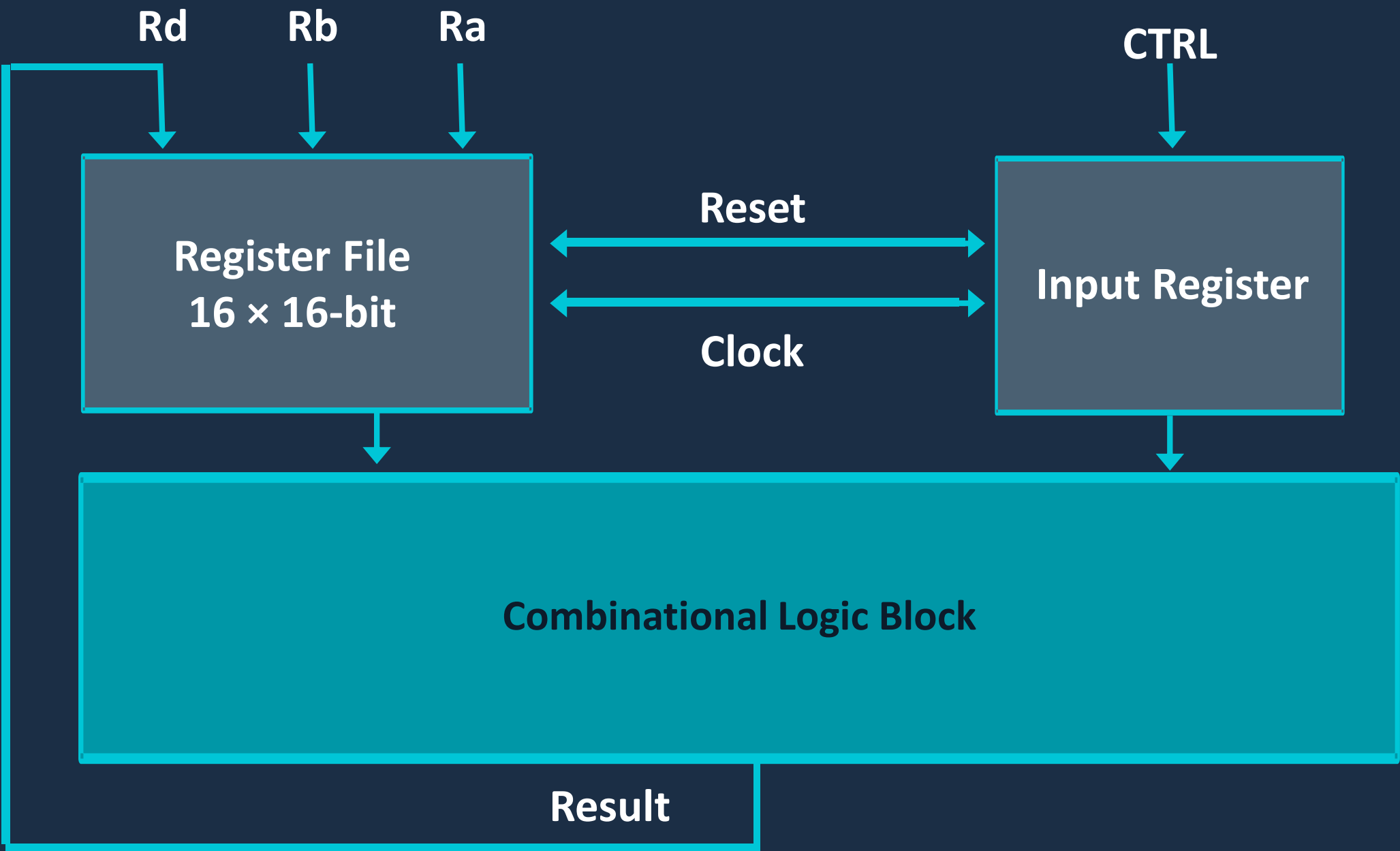


Our Solution: Crypto16

A 16-bit coprocessor in VHDL that handles ALU operations, bit shifts, and S-Box substitution in one integrated datapath.

Implemented entirely in VHDL • Modular design • Full testbench suite

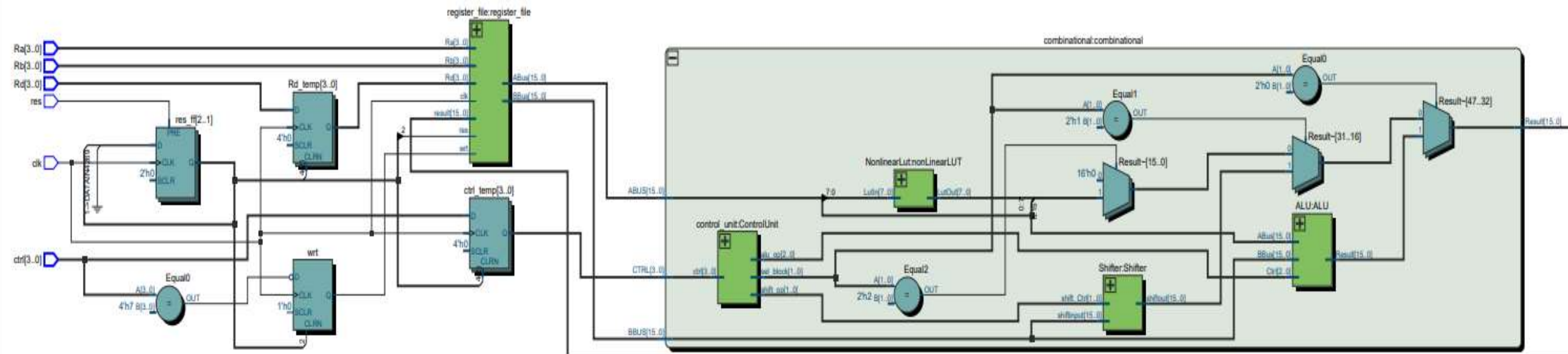
System Architecture



Entity	Type
crypto16_coprocessor	Structural
register_file	Sequential
combinational	Structural + Behavioral
control_unit	Behavioral
ALU	Combinational
shifter	Combinational
NonLinearLut	Combinational
Adder_16bit	Structural

System Architecture

crypto16_coprocessor (Top Level)



Control Unit

The Brain — Decoding 4-bit CTRL into datapath signals

CTRL Range	sel_block	Selected Block	Sub-signal
0000 – 0111	00	ALU	ctrl[2:0] → alu_op (3-bit)
1000 – 1010	01	Shifter	ctrl[1:0] → shift_op (2-bit)
1011	10	NonLinearLut	Direct selection
1100 – 1111	11	NOP / Reserved	Output = 0x0000

Output Signals Generated

- sel_block (2-bit) → ALU / Shifter / LUT / NOP
- alu_op (3-bit) → Which ALU operation
- shift_op (2-bit) → Which shift operation

Key Design Decision

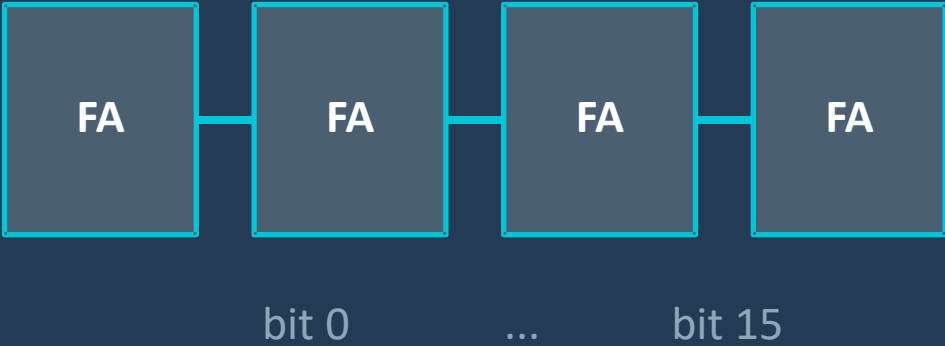
The control unit completely separates instruction decoding from execution. The combinational block never interprets CTRL directly — it only reads pre-decoded sel_block signals.

Arithmetic & Logic Unit (ALU)

alu_op	Operation	Description
000	ADD	A + B via 16-bit ripple-carry adder
001	SUB	A – B (two's complement: A + ~B + 1)
010	AND	Bitwise A AND B
011	OR	Bitwise A OR B
100	XOR	Bitwise A XOR B
101	NOT	Bitwise complement of A
110	MOVE	Pass-through: Output = A

Ripple-Carry Adder

16 Full Adder primitives chained in series.
Carry propagates from bit 0 → bit 15.



SUB uses: A + (NOT B) + 1

Undefined alu_op → Output = 0x0000

Fully Combinational • No clock required • Result available within propagation delay

Register File

16 × 16-bit General-Purpose Registers

- Storage array: reg_data (0 to 15).
- Asynchronous Reads:
ABus = reg[Ra], BBus = reg[Rb] – available immediately, no clock delay.
- Synchronous Write:
On rising clock edge, if wrt = '1', result → reg[Rd].
- Reset Behaviour:
Asynchronous: all 16 registers clear to 0x0000 immediately when res = '1'.

REGISTER ARRAY (16 × 16-bit)

R00 16-bit 0x0000	R01 16-bit 0x0000	R02 16-bit 0x0000	R03 16-bit 0x0000
R04 16-bit 0x0000	R05 16-bit 0x0000	R06 16-bit 0x0000	R07 16-bit 0x0000
R08 16-bit 0x0000	R09 16-bit 0x0000	R10 16-bit 0x0000	R11 16-bit 0x0000
R12 16-bit 0x0000	R13 16-bit 0x0000	R14 16-bit 0x0000	R15 16-bit 0x0000

Shifter Block

Fixed-pattern bit rotations & shifts — fundamental to lightweight ciphers



ROR 8

IN: **FF00**

OUT: **00FF**

Rotate right by 8 bits.
Upper and lower bytes swapped.



ROR 4

IN: **FF00**

OUT: **0FF0**

Rotate right by 4 bits.
Each nibble shifts one position
rightward.



SLL 8

IN: **B38F**

OUT: **8F00**

Logical left shift by 8. Lower byte
zeroed, upper = original lower.

Any other shift_ctrl code → Output = 0x0000 (NOP fallback — no undefined behavior)

Nonlinear LUT — S-Box

The heart of cryptographic security — nonlinear substitution

How It Works

- 1

Take lower 8 bits of ABus
LutIn[7:0]
- 2

Split into two nibbles:
Upper [7:4] → S_Box1
Lower [3:0] → S_Box2
- 3

Each nibble substituted
independently (4-bit → 4-bit)
- 4

Concatenate results → LutOut[7:0]
Upper 8 bits pass through unchanged

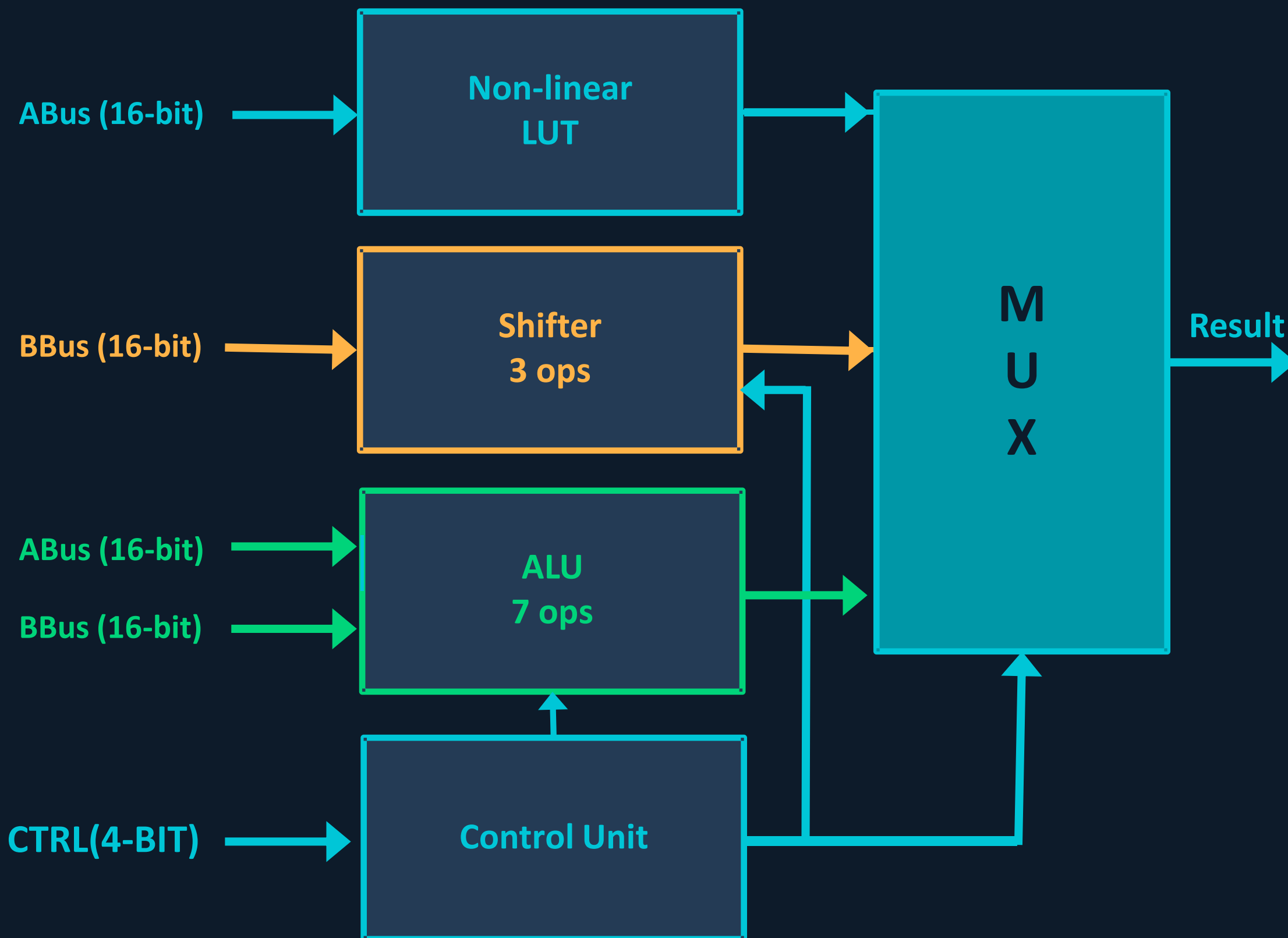
S-Box 1 (Upper Nibble)

In	Out
0	1
1	B
2	9
3	C
4	D
5	6
6	F
7	3
8	E
9	8
A	7
B	4
C	A

S-Box 2 (Lower Nibble)

In	Out
0	F
1	0
2	D
3	7
4	B
5	E
6	5
7	A
8	9
9	2
A	C
B	1
C	3

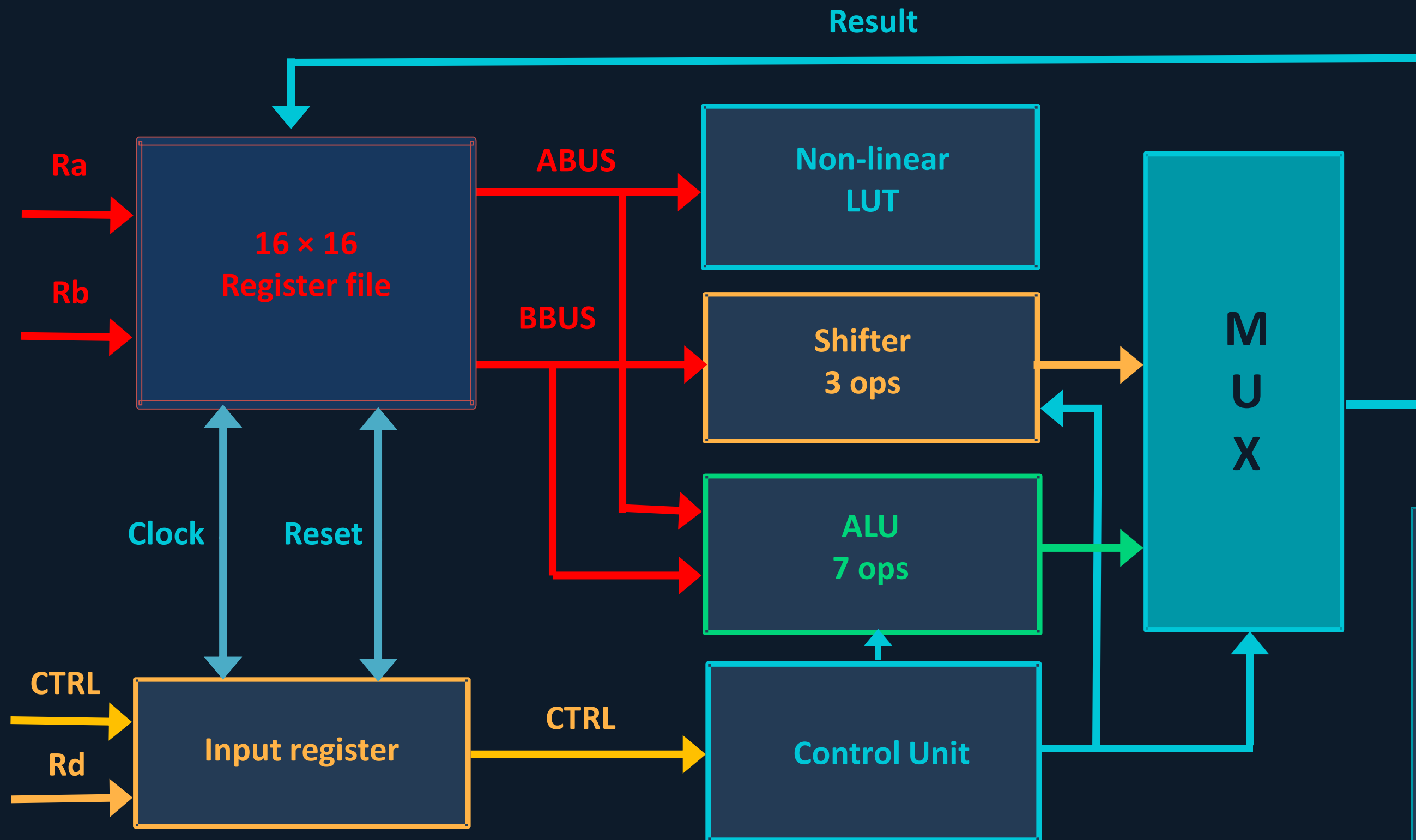
Combinational Block & MUX



Result Source Selection	
sel_block	Source
00	ALU Output
01	Shifter Output
10	LUT Output
11	0x0000 (NOP)

All 4 blocks run in parallel.
The MUX selects only ONE result
based on sel_block from Control Unit.

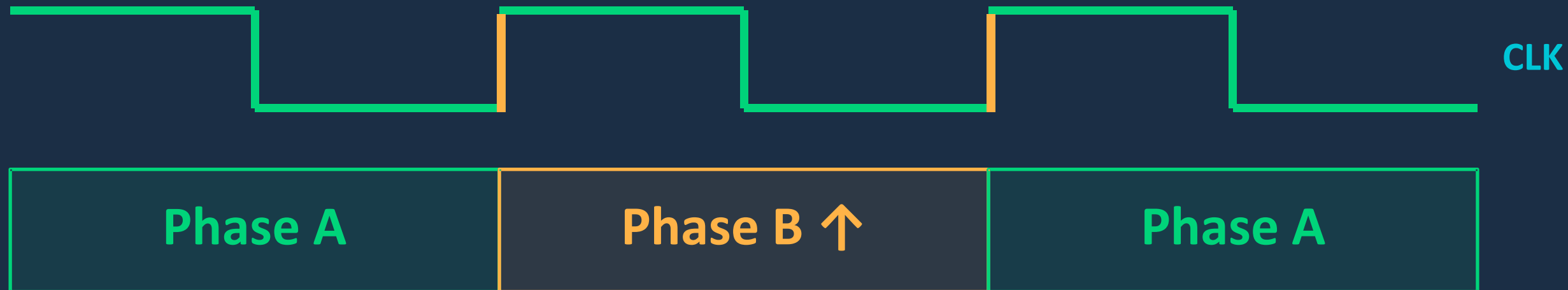
Coprocessor integration



Crypto16 Coprocessor is a 16-bit VHDL datapath with a hybrid reset, combining register storage, control decoding, and ALU, shifter, and LUT execution in one design.

Timing Model

Single-cycle datapath • Asynchronous reads • Synchronous writes



Phase A — Combinational Evaluation

- ABus & BBus read asynchronously from register file
- ALU / Shifter / LUT all compute simultaneously
- MUX selects result based on sel_block
- No clock needed — result available after propagation delay

Phase B — Sequential Commit (Rising Edge)

- Result written to registers[Rd] if wrt = '1'
- ctrl and Rd sampled into ctrl_temp / Rd_temp
- Hybrid reset: async assert, sync deassert (prevents metastability)
- wrt = '0' for ctrl = 0111 (NOP — write disabled)

Verification & Testbench Strategy

Level 1 — Unit Testbenches

- Full_Adder_TB — single-bit carry logic
- Adder_16bit_TB — overflow, 0+0, 0xFFFF+1
- ALU_TB — all 7 ops + edge values
- Shifter_TB — all 3 shifts + undefined codes
→ zero
- NonLinearLut_TB — full 256-value sweep
- register_file_TB — reset, write-enable, read-back

Level 2 — Integration Testbenches

- combinational_TB — MUX switching, CTRL routing across all blocks
- crypto16_coprocessor_TB — end-to-end: write ops, execute, readback

Directed stimulus • Expected outcomes in inline comments • Waveform inspection in Active-HDL

Simulation Results

ALU	
ADD A=0x0032, B=0x001E → 0x0050	✓ PASS
AND A=0xF0F0, B=0xF0F0 → 0x0000	✓ PASS
XOR A=0xAAAA, B=0xAAAA → 0x0000	✓ PASS
NOT A=0x00FF → 0xFF00	✓ PASS

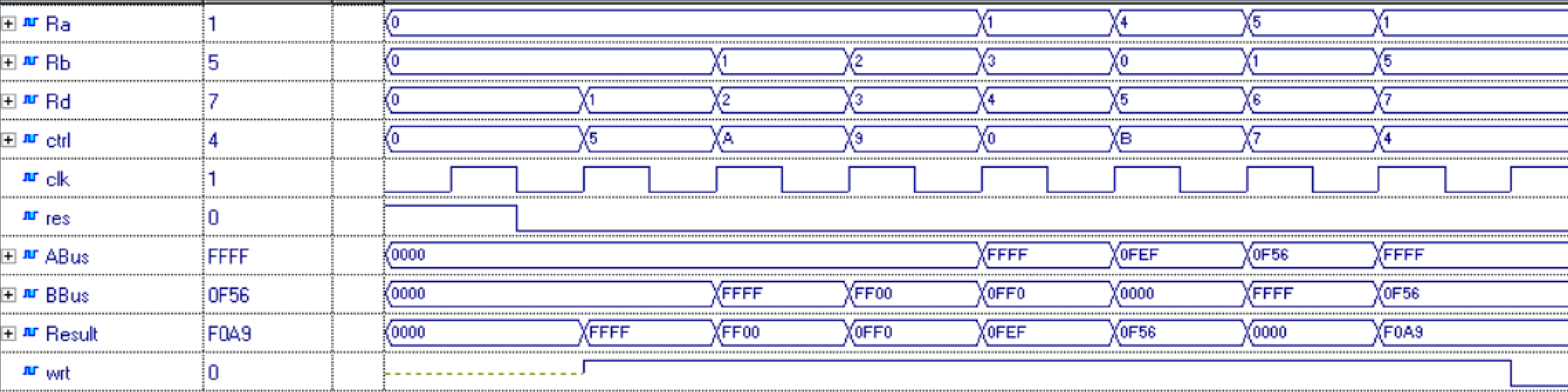
Shifter	
ROR8 A=0xABCD → 0xCDAB	✓ PASS
ROR4 A=0xABCD → 0xDABC	✓ PASS
SLL8 A=0xABCD → 0xCD00	✓ PASS
Undef ctrl=1111 → 0x0000	✓ PASS

LUT (S-Box)	
Input 0x00 LutIn=0x00 → 0x1F	✓ PASS
Input 0x0FB LutIn=0x0B → 0x11	✓ PASS
All 256 values Full sweep → All match table	✓ PASS
Upper 8 bits ABus=0xFF12 → Upper=0xFF pass-through	✓ PASS

Top Level Results

Sample results

CTRL => A (SLL8) Shifter=Shiftin(7 downto 0) & x"00" → 0xFF00	✓ PASS	CTRL => 0 (ALU addition) ABUS + BBUS → 0x0FEF	✓ PASS	CTRL => B (LUT) LutIn=0xEF → 0x0F56	✓ PASS
---	--------	---	--------	---	--------



Design Strengths

Highly Modular



Each block independently instantiable and testable. Swap or upgrade any unit without touching others.

Structurally Explicit



Ripple-carry adder built from primitive full adders — every gate traceable for formal verification.

Clean Separation



Storage, computation, and control are logically partitioned across distinct entities.

Deterministic Reset



All registers clear to 0x0000 on reset. Undefined opcodes produce 0x0000. No undefined behavior.

Three Crypto Classes



ALU + Shifter + S-Box in one integrated 16-bit datapath — covers arithmetic, diffusion, and confusion.

VHDL Best Practices



Uses ieee.numeric_std throughout. Hybrid reset synchronizer prevents metastability.

Conclusion

What We Built

- Fully functional 16-bit cryptographic coprocessor in VHDL
- 3 execution units: ALU (7 ops), Shifter (3 ops), LUT (S-Box)
- 16×16-bit register file with async read / sync write
- Control unit decoding 4-bit opcodes into datapath signals
- Complete testbench suite at unit + integration level

What We Learned

- How dedicated hardware accelerates & secures cryptography
- Structural vs behavioral VHDL — when to use each
- Modular design: independent units → easier testing
- Bottom-up verification: test primitives before integration
- Role of S-Boxes in resisting linear & differential cryptanalysis

Crypto16 bridges basic digital logic and real-world cryptographic hardware design.

Thank You

Questions?

Team

Ahmed Mahmoud Abdel Fattah

Coprocessor Integration, Control Unit, Presentation

Ahmed Salah Abdel Majeed

Shifter, Schematic

Ibraheem Shaaban Abdel Nabi

LUT, Report

Ahmed Abdel Fattah Elmallah

ALU, Schematic

Ahmed Tharwat Salah

Combinational Integration, Presentation

Wateen Walid Mohamed Baree

Register file, Report